

DISCIPLINA: PROGRAMAÇÃO ORIENTADA A OBJETOS

RELATÓRIO TÉCNICO:

RPG Manager: The Witcher
Sistema de Gerenciamento de RPG
Decisões de Design

DISCENTES

Maria Eduarda Cardoso de Oliveira RA: a2533880

Arieli Leandro Gutierrez RA: a2669447

DOCENTE

Prof. Rafael Gomes Mantovani

Sumário

1	Tema e Motivação	2
2	Decisões sobre as Classes	2
2.1	Hierarquia de Jogador e Raças	2
2.2	Classe Profissão	2
2.3	Interfaces ICombate e ICuravel	3
2.4	Encapsulamento	3
3	Por que as Variáveis Vieram do Livro	3
4	Padrões de Projeto	4
4.1	Factory Method: JogadorFactory, ProfissaoFactory e MonstroFactory . . .	4
4.2	Template Method: aumenta_nivel_habilidade	4
4.3	Abstract Class / Interface	4
5	Interface Gráfica	4
5.1	Decisão de Tecnologia	4
5.2	Arquitetura da Interface	4
5.3	Decisões de UX	5
6	Testes Automatizados	5
6.1	Estratégia de Testes	5
6.2	Cobertura	5
7	Diagrama UML	5
	Conclusão	6

1 Tema e Motivação

O projeto **RPG Manager** foi desenvolvido como sistema de gerenciamento de personagens para um jogo de RPG de mesa, tendo como tema o universo de *The Witcher*, criado pelo escritor polonês Andrzej Sapkowski e popularizado pela série de jogos da CD Projekt RED.

A escolha do tema foi motivada por três razões principais:

- **Riqueza de entidades modeláveis:** o universo Witcher possui raças, profissões, habilidades e monstros bem definidos em material canônico, o que facilita a modelagem orientada a objetos sem necessidade de inventar regras arbitrárias.
- **Material de referência:** o *The Witcher TRPG Sourcebook* (livro de RPG de mesa oficial) define atributos numéricos, habilidades específicas por raça e profissão, e mecânicas de combate que serviram de base direta para as classes implementadas.
- **Alinhamento com os requisitos do projeto:** a estrutura do jogo (hierarquia de personagens, sistema de nível e combate por turnos) mapeia naturalmente para os conceitos de herança, polimorfismo e padrões de projeto exigidos na disciplina.

2 Decisões sobre as Classes

2.1 Hierarquia de Jogador e Raças

A classe central do sistema é **Jogador**, que implementa as interfaces abstratas **ICombate** e **ICuravel**. Essa decisão separa as **capacidades** do personagem (combater, curar-se) de sua **identidade** (raça, profissão).

As raças foram modeladas como **subclasses concretas** de **Jogador**: **Humano**, **Bruxo**, **Anao** e **Elfo**. Cada raça herda todos os atributos base e adiciona habilidades exclusivas.

A decisão de usar **raças como subclasses** (e não como atributo de texto) foi deliberada: permite que cada raça tenha métodos e atributos próprios, aproveitando herança e polimorfismo. Isso difere de uma simples string `raca = "Humano"`, que não carrega comportamento.

A classe **Jogador** é **abstrata** (possui `@abstractmethod inicializa_jogador`), impedindo instânciação direta e forçando o uso das subclasses de raça.

2.2 Classe Profissão

A **Profissão** foi modelada como uma hierarquia **separada** de **Jogador**, sendo associada por composição: cada jogador possui uma referência a um objeto **Profissao**.

Essa decisão foi tomada porque no universo Witcher a profissão é independente da raça: um Humano pode ser Mago ou Criminoso; um Anão pode ser Cavaleiro ou Comerciante. Se profissão fosse herança de raça, seria necessário criar combinações como **HumanoMago**, **HumanoCriminoso**, resultando em explosão combinatória de classes.

Profissao é abstrata com `@abstractmethod inicializa_profissao`. As profissões concretas implementadas são: **Mago**, **Bardo**, **Artesao**, **Criminoso**, **Doutor**, **Cavaleiro**, **Comerciante**, **Sacerdote** e **Desempregado**.

2.3 Interfaces ICombate e ICuravel

As interfaces foram definidas como classes abstratas Python com `@abstractmethod`, seguindo o princípio de separação de interfaces.

Essa estrutura obriga a classe `Jogador` a implementar `atacar()` e `curar()`, estabelecendo um contrato formal de comportamento. Já o método abstrato `inicializa_jogador()` da própria `Jogador` obriga cada raça concreta a fornecer sua implementação.

2.4 Encapsulamento

O projeto segue a convenção Python de encapsulamento:

- `_atributo` (um underscore): protegido, acessível em subclasses
- `__atributo` (dois underscores): privado, com name mangling pelo interpretador, não acessível diretamente de fora da classe

Os atributos são acessados via **getters** e **setters** explícitos, com validação incorporada nos setters.

3 Por que as Variáveis Vieram do Livro

As habilidades de cada raça e profissão foram extraídas diretamente do *The Witcher TRPG Sourcebook*, que define um sistema de atributos numéricos para o jogo de mesa oficial. Isso garante consistência temática e justifica as escolhas de design sem necessidade de arbitrariedade.

Tabela 1: Habilidades específicas por raça (fonte: Witcher TRPG Sourcebook)

Raça	Habilidades exclusivas
Humano	Sedução, Persuasão, Teimosia
Bruxo	Reflexos Relâmpagos (Carisma fixo em 0, conforme o cânon)
Anão	Armadura, Dedução
Elfo	Artesanato, Artilharia de Arcos, Sintonia da Natureza

Tabela 2: Habilidades específicas por profissão (fonte: Witcher TRPG Sourcebook)

Profissão	Habilidades
Mago	Criar Ritual, Lançar Feitiço, Educação, Resistir Magia
Bardo	Etiqueta Social, Ludibriar, Sabedoria das Ruas
Artesão	Educação, Criação, Negociação, Físico
Criminoso	Arrombar Fechaduras, Falsificação, Atletismo, Sabedoria das Ruas
Doutor	Carisma, Educação, Coragem, Alquimia
Cavaleiro	Coragem, Intimidação, Sobrevivência, Esquivar
Comerciante	Carisma, Educação, Negócios, Persuasão
Sacerdote	Criar Ritual, Coragem, Ensinar, Lançar Feitiço
Desempregado	Sobrevivência, Consciência

A imunidade de cada raça também é canônica: Humano (5), Anão (7), Elfo (8) e Bruxo

(13), o valor mais alto. Isso reflete que os Bruxos passam por mutações que os tornam resistentes a venenos e doenças.

4 Padrões de Projeto

4.1 Factory Method: JogadorFactory, ProfissaoFactory e MonstroFactory

O padrão **Factory** é utilizado em três classes do sistema, centralizando a criação de objetos e desacoplando o código cliente da lógica de instanciação.

4.2 Template Method: aumenta_nivel_habilidade

O padrão **Template Method** é implementado no método `aumenta_nivel_habilidade()`: a classe `Jogador` define o algoritmo base (upar habilidades comuns: brigar, apostar, força, inteligência) e cada subclasse de raça **estende** esse comportamento adicionando suas habilidades exclusivas via `super()`.

4.3 Abstract Class / Interface

As classes `Profissao` e `Jogador` são abstratas (utilizam o módulo `abc` do Python), e `ICombate` e `ICuravel` funcionam como interfaces. Isso garante que as subclasses concretas implementem os métodos obrigatórios, estabelecendo contratos formais de comportamento.

5 Interface Gráfica

5.1 Decisão de Tecnologia

A interface gráfica foi desenvolvida em **PyQt5**, biblioteca Python que encapsula o framework Qt. A escolha se deu por:

- Compatibilidade com Python (linguagem do projeto)
- Suporte a estilização via *Qt Style Sheets* (QSS), similar ao CSS
- Widgets ricos (sliders, combo boxes, text areas) adequados ao domínio do jogo

5.2 Arquitetura da Interface

A interface foi implementada em arquivo separado (`player_inventory_interface.py`), que **importa as classes** do arquivo de lógica sem modificá-lo (`from player_inventory import *`).

Essa decisão mantém a separação entre lógica de negócio e camada de apresentação, seguindo o princípio de responsabilidade única.

A interface utiliza `QStackedWidget` para alternar entre a tela de criação de personagem e a tela de jogo sem abrir novas janelas.

5.3 Decisões de UX

- **Sliders (barras deslizantes)** em vez de campos numéricos para atributos com limites definidos (0–13 para habilidades, 0–300 para HP): mais intuitivo e evita entrada de valores inválidos.
- **Tema visual** escuro com dourado (#c8a44d), referenciando a estética visual da série Witcher.
- **Log de eventos** textual na tela de jogo, exibindo o resultado de cada rodada de combate (dano causado, dano recebido, HP restante).
- **Painel de status** atualizado automaticamente após cada ação, mostrando nome, raça, profissão, HP e nível do personagem.

6 Testes Automatizados

6.1 Estratégia de Testes

Os testes foram implementados com o módulo `unittest` (equivalente Python ao Google Test em C++, como exigido pela especificação do projeto).

A estratégia adotada foi testar a **camada de lógica** diretamente, sem passar pela interface. Isso é necessário porque os métodos de menu interativo da interface de terminal utilizam `input()`, que não é testável por unidade sem simulação de entrada.

6.2 Cobertura

Foram implementados **49 testes** organizados em 7 grupos:

Tabela 3: Grupos de testes e o que cobrem

Grupo	Cobertura
TesteCriacaoPersonagem	Criação das 4 raças, verificação de atributos e imunidade
TesteValidacoes	Nome vazio, HP inválido, atributos negativos
TestePontos	Conversão de nível em pontos, sobra correta
TesteProfissoes	Criação das 9 profissões, setters e getters
TesteSkillsRaca	Habilidades específicas de cada raça
TesteMonstro	Dano, HP não negativo, morte, Factory
TesteCombate	Dano no monstro, dano recebido, XP ao matar, cura

A cobertura medida foi de **46%** do arquivo de lógica. O código não coberto corresponde majoritariamente aos métodos de menu interativo (`umenta_nivel_habilidade` e `umenta_nivel_habilidade_profissao`) que dependem de `input()` e, por isso, não são testáveis por unidade. O código lógico crítico está integralmente coberto.

7 Diagrama UML

O diagrama UML completo do projeto foi elaborado no **PlantUML** e está disponível no relatório de especificações de design separado. A estrutura principal pode ser resumida

como:

- ICombate e ICuravel $\leftarrow$ interfaces abstratas
- Jogador implementa ICombate e ICuravel
- Humano, Bruxo, Anao, Elfo herdam de Jogador
- Profissao (abstrata) $\leftarrow$ 9 profissões concretas herdam
- Jogador tem composição com Profissao
- Monstro é criado por MonstroFactory
- JogadorFactory e ProfissaoFactory instanciam as subclasses concretas

Conclusão

O projeto RPG Manager implementou com sucesso um sistema de gerenciamento de personagens orientado a objetos, aplicando herança, polimorfismo, encapsulamento, composição e padrões de projeto (Factory, Template Method e Abstract Class). O tema do Witcher permitiu decisões de design fundamentadas em material canônico, tornando o sistema consistente e extensível.